

Necessità → Rappresentare l'esecuzione di funzioni:

$f: \text{Input} \rightarrow \text{Output}$

⇒ vogliono descrivere COME calcolare f ovvero vogliono rappresentare ALGORITMI

→ sequenza ordinata di passi discreti che descrivono come calcolare f

strategia per il calcolo di f

⇒ Abbiamo bisogno di uno strumento che permetta di manipolare dati descrivendo Algoritmi:

PROGRAMMA

⇒ linguaggi di programmazione: linguaggi formali che permettono di scrivere programmi (PL)

PROGRAMMI: Frasi ben formate di un PL, sono una rappresentazione FINITA di una sequenza potenzialmente INFINITA di effetti su una macchina

APPLICAZIONI

- Applicazioni: scientifiche FORTRAN
- Applicazioni: economiche COBOL →
 - poco efficiente
 - poco efficace
 - vecchio
 - usato molto perché cambiare prenderebbe tanto tempo
- Applicazioni: AI LISP
- Applicazioni: sistemi C
- Software Web JAVA

LIVELLO

- BASSO LIVELLO → linguaggio macchina
 - ALTO LIVELLO → paradigmi
 - imperativo (cella memoria) } funzionali
 - funzionale (matematico) } impuri
 - logico → PROLOG
- procedurale → imperativo/strutturato (procedurale senza salti)
- OOD → JAVA (incapsulamento in oggetti)
- dichiarative → logico/funzionali (bei matematiche)

GENERAZIONALI

- I: Hardware proprietari
- II: Assembly
- III: Alto livello: uso linguaggio naturale
- IV: SQL
- V: Applicazioni: danno risposta a problemi specifici

CARATTERISTICHE

- sequenziali
- concorrenti (multi thread)
- parallelo (non c'è bisogno di comunicazione)
- distribuiti
- oggetti
- scripting (interpretati)
- ...

LEGGIBILITÀ

WRITABILITY

AFFIDABILITÀ

completo

- quantità limitata di costrutti
- poca molteplicità di elementi
- poco overloading di operatori

Type checking
Gestione eccezioni

Aliasing (negativo) → nomi diversi stesso cella

Ortogonalità
Tipi di dati
Sintassi

Astrazione (OO)
Espressività

OO = Object Oriented

IMPLEMENTARE UN PL

- Architettura della macchina
 - Metodologie di progettazione di programmi:
 - dati e programmi in memoria (separata dalla CPU)
 - istruzioni e dati che viaggiano
- [

program counter
repeat forever
fetch instruction nella memoria usando program counter
aggiorniamo program counter
decode instruction
execute instruction
end repeat

]

Implementare PL significa costruire la macchina astratta per il PL



PL ma linguaggio L con i suoi costrutti sintattici
↓
programma (frase ben formata nel PL)

Per esempio costruiamo la macchina astratta per L

MACCHINA ASTRATTA

Dato L (linguaggio di programmazione)
 M_L macchina astratta di L o un insieme di strutture dati e algoritmi che permettano di eseguire programmi scritti in L

LINGUAGGIO MACCHINA

Dato M macchina astratta il linguaggio L_M è chiamato linguaggio macchina di M ed è l'insieme di tutte le stringhe eseguibili (riconoscibili) da M

Implementare un PL → Realizzare M_L

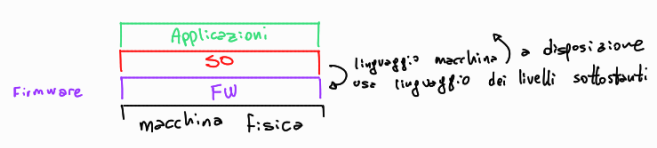
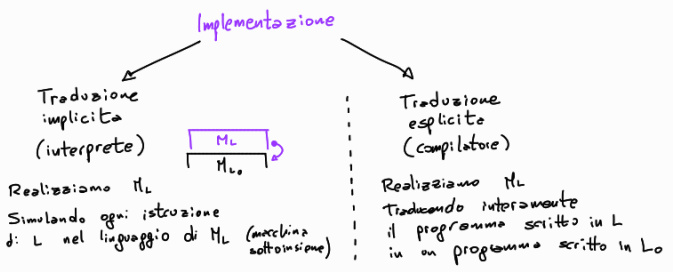


IMMAGINE SLIDE



NOTAZIONE

$Prog^L$ = insieme di programmi scritti in L
 D = insieme dei dati
 $P^L \in Prog^L$ programma scritto in L
 $P^L : D \rightarrow D$
input D P^L (input)
Esecuzione di P^L sul dato input restituisce il dato output

INTERPRETE

Un interprete per il linguaggio L, scritto in L_0
(interprete scritto in un linguaggio implementato da M_{L_0} sottostante)
è un programma

$$I^{L_0} : Prog^L \times D \rightarrow D$$
$$\forall input \in D. I^{L_0}(P^L, input) = output$$
$$\forall P^L \in Prog^L$$

MACCHINA UNIVERSALE

